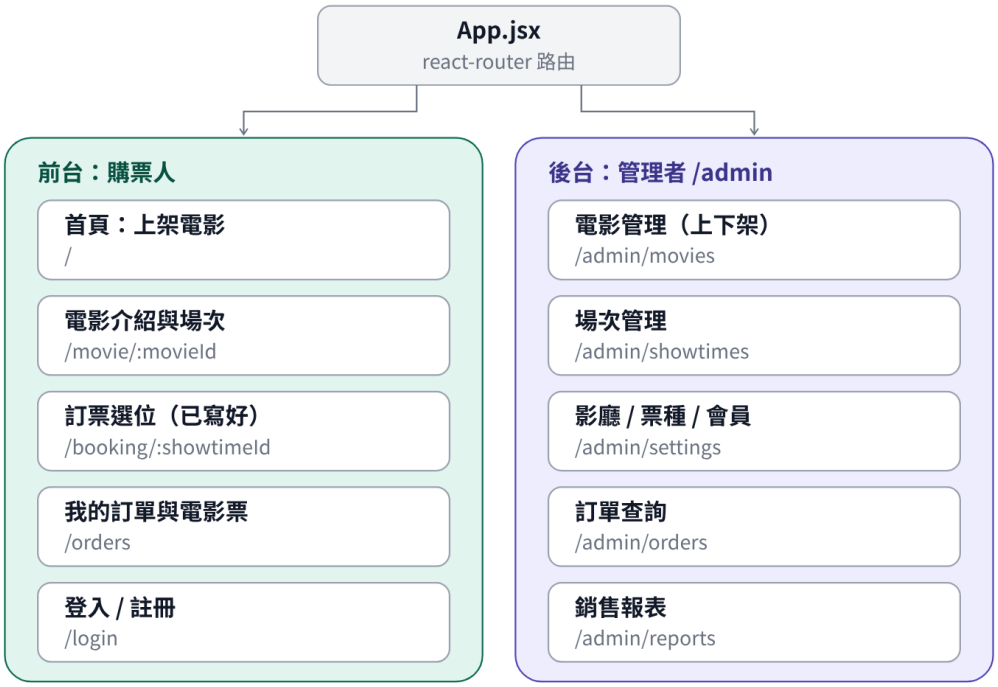
movie 電影購票系統 前端網站架構與網頁示意圖

2026-11-02

總覽

前端用 React（Vite 建立）+ react-router 做頁面切換，所有資料都透過後端 API 取得；後端在 `http://localhost:8080`，前端開發伺服器在 `http://localhost:5173`，後端已開放 CORS。

網站分成兩區：前台給購票人（看電影、訂票、看自己的訂單），後台給管理者（維護電影、場次、影廳、票種，查訂單與報表）。



網站架構：App.jsx 用 react-router 分成前台 5 頁與後台 5 頁

路由與頁面

區域	網址	頁面元件	呼叫的 API
前台	`/`	`HomePage`	`GET /api/movie?page=&size=`
前台	`/movie/:movieId`	`MovieDetailPage`	`GET /api/movie/{id}`、`GET /api/showtime/upcoming?movieId=`
前台	`/booking/:showtimeId`	`BookingPage`（已寫好）	`GET /api/showtime/{id}`、`GET /api/hall/{id}`、`GET /api/ticket-type`、`GET /api/ticket?showtimeId=`、`POST /api/orders`
前台	`/orders`	`MyOrdersPage`	`GET /api/orders?memberId=`、`GET /api/orders/{id}`

區域	網址	頁面元件	呼叫的 API
前台	/login	LoginPage	POST /api/member（註冊）、POST /api/member/login（登入）
後台	/admin/movies	AdminMoviesPage	GET /api/movie、GET /api/movie/inactive、POST /api/movie/add、PUT /api/movie/{id}
後台	/admin/showtimes	AdminShowtimesPage	GET/POST/PUT/DELETE /api/showtime
後台	/admin/settings	AdminSettingsPage	/api/hall、/api/ticket-type、/api/member
後台	/admin/orders	AdminOrdersPage	GET /api/orders、GET /api/ticket?orderId=
後台	/admin/reports	AdminReportsPage	GET /api/admin/report/sales?period=、GET /api/admin/report/movie-sales

資料夾結構

```
movie-frontend/
├─ index.html
├─ package.json
└─ src/
   ├─ main.jsx                進入點：把 <App /> 放進網頁
   ├─ App.jsx                 路由設定（react-router）
   ├─ api.js                  所有呼叫後端的函式集中在這裡
   ├─ seatUtils.js            座位規則（每排幾個座位）
   ├─ booking.css             訂票頁、電影票的樣式
   ├─ components/             多個頁面共用的元件
   │  ├─ Header.jsx             上方導覽列
   │  ├─ MovieCard.jsx          首頁的海報卡片
   │  ├─ SeatMap.jsx            座位圖（已寫好）
   │  ├─ TicketList.jsx         電影票（已寫好）
   │  └─ Pagination.jsx         分頁按鈕
   └─ pages/                  每個網址對應一個頁面
      ├─ HomePage.jsx
      ├─ MovieDetailPage.jsx
      ├─ BookingPage.jsx      （已寫好）
      ├─ MyOrdersPage.jsx
      ├─ LoginPage.jsx
      └─ admin/
         ├─ AdminMoviesPage.jsx
         ├─ AdminShowtimesPage.jsx
         ├─ AdminSettingsPage.jsx
         ├─ AdminOrdersPage.jsx
         └─ AdminReportsPage.jsx
```

App.jsx 路由範例

先安裝 `npm install react-router-dom`，再把網址對應到頁面元件。

```
import { BrowserRouter, Routes, Route, useParams } from "react-router-dom";
import HomePage from "./pages/HomePage";
```

```
import MovieDetailPage from "./pages/MovieDetailPage";
import BookingPage from "./pages/BookingPage";
import AdminReportsPage from "./pages/admin/AdminReportsPage";

// 網址上的 :showtimeId 用 useParams() 取出來，再傳給 BookingPage
function BookingRoute() {
  const { showtimeId } = useParams();
  return <BookingPage showtimeId={Number(showtimeId)} memberId={1} />;
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/movie/:movieId" element={<MovieDetailPage />} />
        <Route path="/booking/:showtimeId" element={<BookingRoute />} />
        <Route path="/admin/reports" element={<AdminReportsPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

`memberId={1}` 是暫時寫死的測試會員。後端已經有登入 API（`POST /api/member/login`），做好 LoginPage 後，把登入回傳的 `memberId` 存在 localStorage，這裡再改成從 localStorage 取得。

網頁示意圖

首頁 /

列出上架中的電影海報，每張卡片顯示片名，點卡片進入電影介紹。資料來自 `GET /api/movie`，回應的 `content` 陣列用 `.map()` 轉成卡片；圖片網址要在前面加上 `http://localhost:8080`。



首頁示意圖

電影介紹與場次 /movie/:movieId

上半部是海報與電影資料，下半部列出這部電影的場次按鈕；點場次就跳到 `/booking/{showtimeId}`。這頁要呼叫兩支 API：`GET /api/movie/{id}` 和 `GET /api/showtime/upcoming?movieId={id}`。場次 API 只回今天以後的場次，已依日期、時間排序，回傳的是陣列（不是分頁格式），沒有場次時是 `[]`。



電影介紹與場次示意圖

訂票選位 /booking/:showtimeId

座位三種狀態：白色未售、藍色預訂、灰色已售。按一下未售座位變預訂，再按一下取消；每個預訂座位可選票種。按「確認訂票」呼叫 `POST /api/orders`，成功後座位變已售，畫面顯示電影票，1 個座位 1 張票，可列印。這頁的程式碼已經寫好（`movie-booking-react-src.zip`）。



訂票選位示意圖

我的訂單與電影票 /orders

列出登入會員的訂單，每筆顯示片名、廳名、放映日期時間、金額，展開後顯示每張票的座位與票種。資料來自 `GET /api/orders?memberId=`，每筆訂單已經帶有 `movieTitle`、`hallName`、`showDate`、`startTime` 和 `tickets`，不用再另外查場次。沒有訂單時這支 API 回 404，畫面要顯示「目前沒有訂單」而不是錯誤。

登入 / 註冊 /login

登入呼叫 POST `/api/member/login`（帳號、密碼），成功回 200 + 會員資料，把 `memberId`、`name` 存到 localStorage，Header 顯示會員名稱；失敗回 401「帳號或密碼錯誤」。註冊呼叫 POST `/api/member`，帳號重複回 409。

銷售報表 /admin/reports

上方切換本週 / 本月，並可輸入片名查單一電影；中間是訂單數、票數、實收三個數字，下方是各電影的銷售張數與金額。資料來自 GET `/api/admin/report/sales?period=week|month` 和 GET `/api/admin/report/movie-sales?title=`。



銷售報表示意圖

每個頁面的寫法

每一頁都是同一個模式，學會一頁，其他頁照著做：

1. `useState` 宣告要存的資料，例如 `const [movies, setMovies] = useState([]);`
2. `useEffect` 在進入頁面時呼叫 API，拿到資料後 `setMovies(...)`
3. 在 `return` 裡用 `.map()` 把陣列轉成畫面元素
4. 按鈕、輸入框用 `onClick`、`onChange` 改變 state，畫面會自動更新

首頁範例：

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { API_BASE } from "../api";

export default function HomePage() {
  const [movies, setMovies] = useState([]);

  useEffect(() => {
    fetch(`${API_BASE}/api/movie?page=1&size=12`)
      .then((res) => res.json())
      .then((page) => setMovies(page.content));
  }, []);

  return (
```

```
    <div className="movie-grid">
      {movies.map((m) => {
        const poster = m.images.find((i) => i.imageType === "POSTER");
        return (
          <Link to={`/movie/${m.movieId}`} key={m.movieId} className="movie-card">
            {poster && <img src={API_BASE + poster.imageUrl} alt={m.title} />}
            <div>{m.title}</div>
          </Link>
        );
      })}
    </div>
  );
}
```

建議撰寫順序

順序	頁面	原因
1	訂票選位	程式碼已寫好，先跑起來，確認前後端能互通
2	首頁	只有一支 GET API，最適合練習 useState / useEffect / map
3	電影介紹與場次	練習兩支 API、網址參數 useParams、Link 跳頁
4	登入 / 註冊	練習表單送出 POST、處理 401 錯誤、用 localStorage 保存 memberId
5	我的訂單	練習用登入的 memberId 查資料、巢狀資料（訂單裡的票）
6	後台電影管理	練習表單與 multipart 上傳（FormData）
7	後台其他頁、銷售報表	都是同樣模式的 CRUD 與查詢

待決定事項

- 座位規則：資料表只有「第一排座位數」，目前 `seatUtils.js` 假設每往後一排多 1 個座位；規則確定後，後端 `OrdersService` 也要加上同樣的座號上限檢查。
- 登入與權限：會員登入 API 已完成，前端用 localStorage 保存 `memberId`；但後端沒有 Session / Token 驗證，`memberId` 由前端傳入，`/api/admin/...` 也還沒有管理者登入與權限保護。